

Software Requirements Specification (SRS)

Eventify: Microservices-Based Event Management System

1. Introduction

1.1 Purpose

The system allows users to browse events, reserve seats, manage bookings, and receive notifications. Administrators can create and manage events, monitor bookings, and manage users. The system is designed following clean architecture principles, microservices architecture, Aspect-Oriented Programming (AOP), and RESTful API design.

1.2 Scope

Eventify is an online event management system that supports:

- User authentication and authorization using JWT
- Event browsing and searching
- Event booking and cancellation
- Notification management
- Administrative event management
- Microservices communication
- API Gateway routing and security
- Dockerized deployment
- Service discovery using Eureka Server

The system is composed of multiple independent services:

- Auth Service
- Event Service
- Booking Service
- Notification Service
- API Gateway
- Eureka Discovery Server

1.3 Technologies Used

Technology	Purpose
Spring Boot	Backend framework
Spring Security	Authentication and authorization
JWT	Secure token-based authentication
Spring Cloud Gateway	API Gateway
Eureka Server	Service discovery
Spring Data JPA	Database persistence
MySQL	Database
Docker	Containerization
AOP	Cross-cutting concerns
Postman	API testing

2. System Overview

Eventify follows a microservices architecture where each service is independently responsible for a specific domain.

The architecture consists of:

- API Gateway for centralized routing and security

- Independent microservices with separate databases
- Service discovery using Eureka
- JWT authentication and role-based authorization
- AOP logging for monitoring and debugging

3. Functional Requirements

3.1 Authentication Module

1 User Registration

The system shall allow guests to register using:

- Full name
- Email
- Password

2 User Login

The system shall authenticate users using email and password.

3 JWT Authentication

The system shall generate JWT tokens after successful authentication.

4 Role-Based Authorization

The system shall support:

- USER role
- ADMIN role

5 User Profile

Authenticated users shall be able to retrieve their profile information.

3.2 Event Module

6 View Events

Users shall be able to view all available events.

7 View Event Details

Users shall be able to view detailed event information.

8 Search Events

Users shall be able to search and filter events.

9 Create Event

Admins shall be able to create new events.

10 Update Event

Admins shall be able to update existing events.

11 Delete Event

Admins shall be able to delete events.

12 Seat Reservation

The system shall reserve seats during booking.

13 Seat Release

The system shall release seats after booking cancellation.

3.3 Booking Module

14 Create Booking

Authenticated users shall be able to reserve event seats.

15 View User Bookings

Users shall be able to view their bookings.

16 View All Bookings

Admins shall be able to monitor all bookings.

17 Cancel Booking

Users shall be able to cancel bookings.

18 Booking Validation

The system shall prevent overbooking.

3.4 Notification Module

19 Create Notifications

The system shall create notifications after booking operations.

20 View Notifications

Users shall be able to retrieve notifications.

21 Mark Notifications as Read

Users shall be able to mark notifications as read.

4. Non-Functional Requirements

4.1 Security Requirements

- JWT-based authentication shall be implemented.
- Passwords shall be encrypted using PasswordEncoder.
- Role-based authorization shall be enforced.
- Protected APIs shall require valid JWT tokens.
- API Gateway shall validate requests before forwarding them.

4.2 Performance Requirements

- API response time should be less than 2 seconds under normal load.
- Services should support concurrent requests.
- Microservices should operate independently.

4.3 Scalability Requirements

- Services shall be independently deployable.
- Each service shall use a separate database.
- The architecture shall support horizontal scaling.

4.4 Availability Requirements

- Service discovery shall be handled using Eureka Server.
- Gateway routing shall provide centralized access management.

4.5 Maintainability Requirements

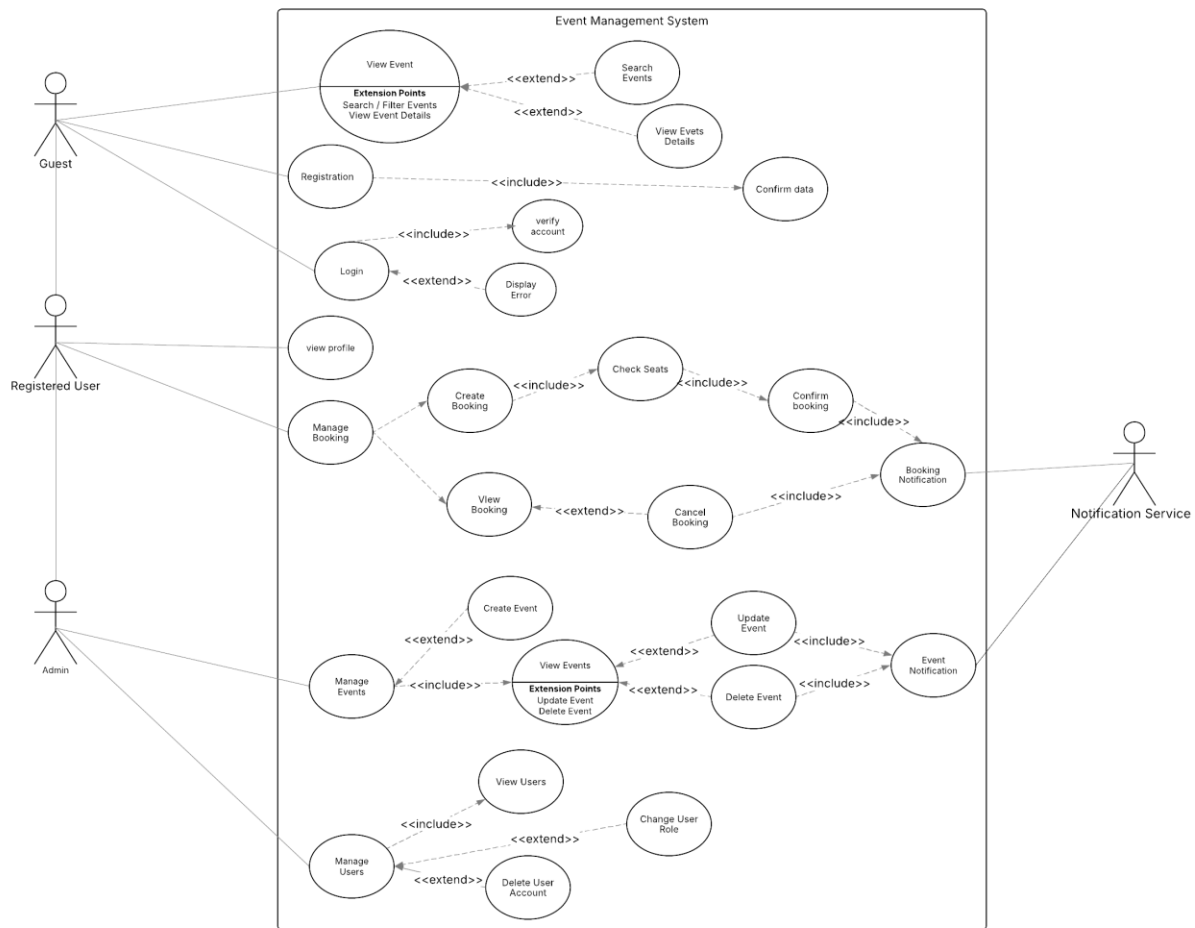
- The system shall follow layered architecture:
 - Controller Layer
 - Service Layer
 - Repository Layer
 - Entity Layer
- SOLID principles shall be applied.
- Repository Pattern shall be used.
- DTO Pattern shall be used.
- AOP shall separate logging concerns from business logic.

5. System Architecture

The system follows a microservices architecture consisting of:

Service	Responsibility
API Gateway	Request routing and JWT validation
Auth Service	Authentication and user management
Event Service	Event management
Booking Service	Booking operations
Notification Service	Notifications
Eureka Server	Service discovery

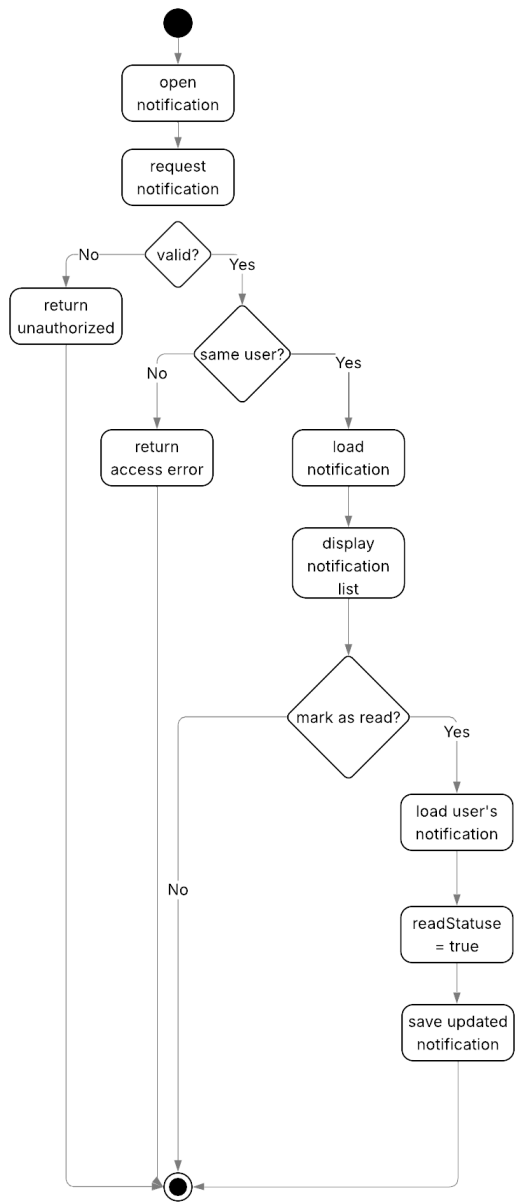
6. Use Case Diagram

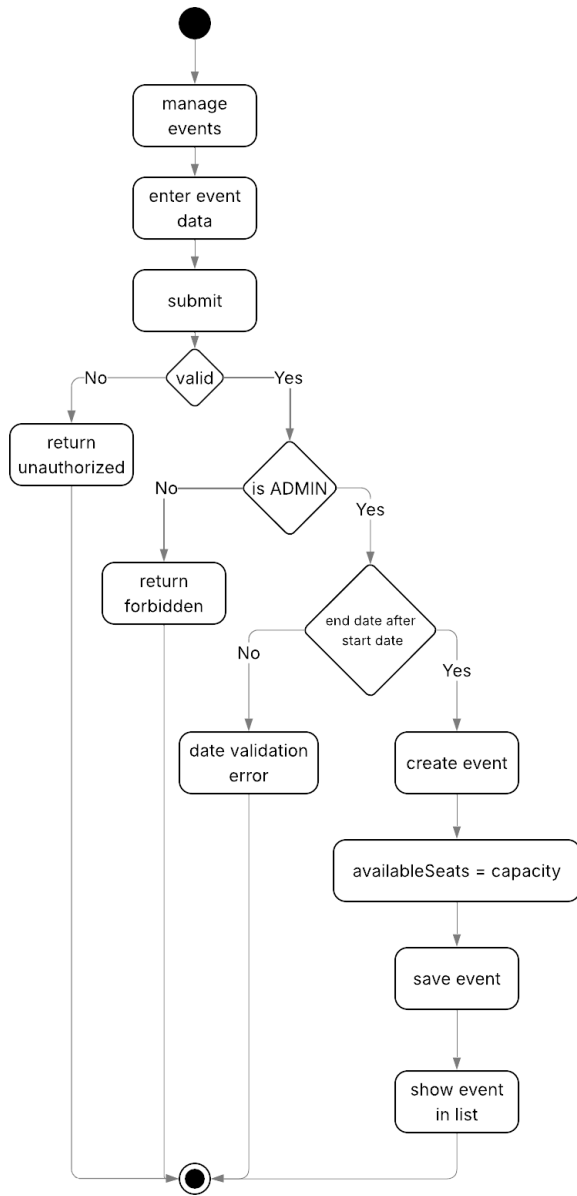


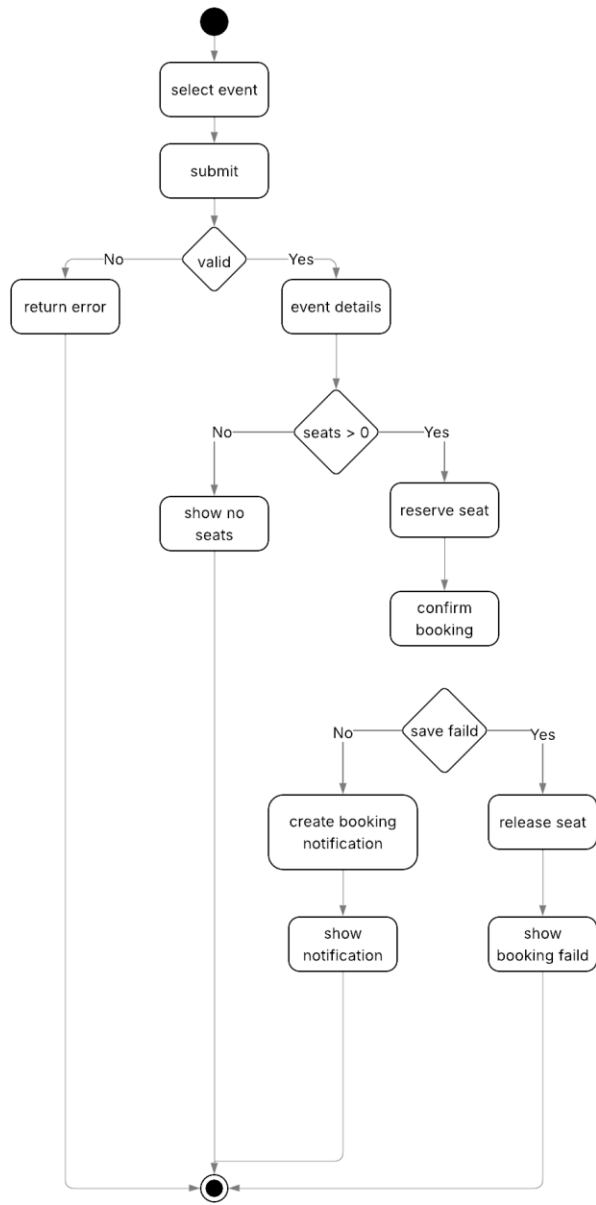
7. Activity Diagrams

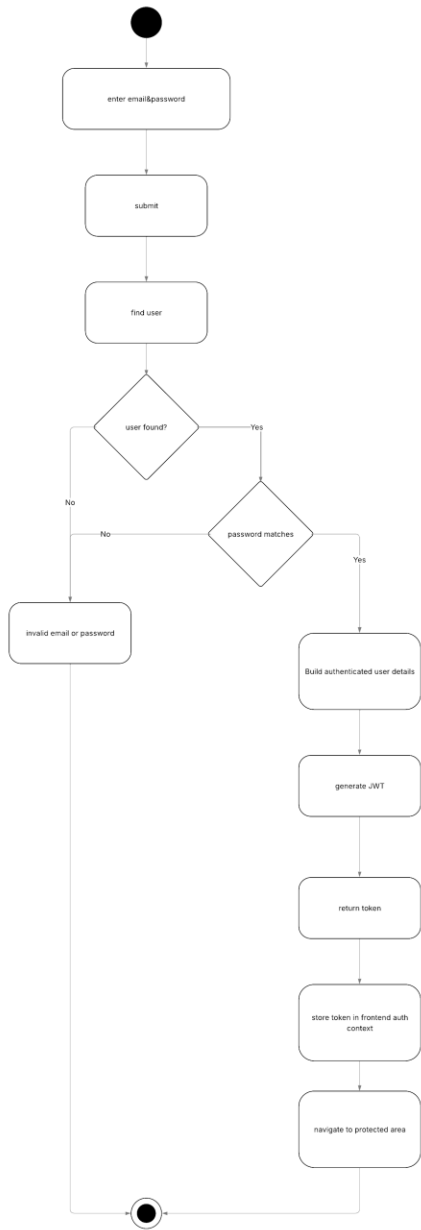
Included Activity Diagrams

- Authentication Activity Diagram
- Events Activity Diagram
- Booking Activity Diagram
- Notifications Activity Diagram





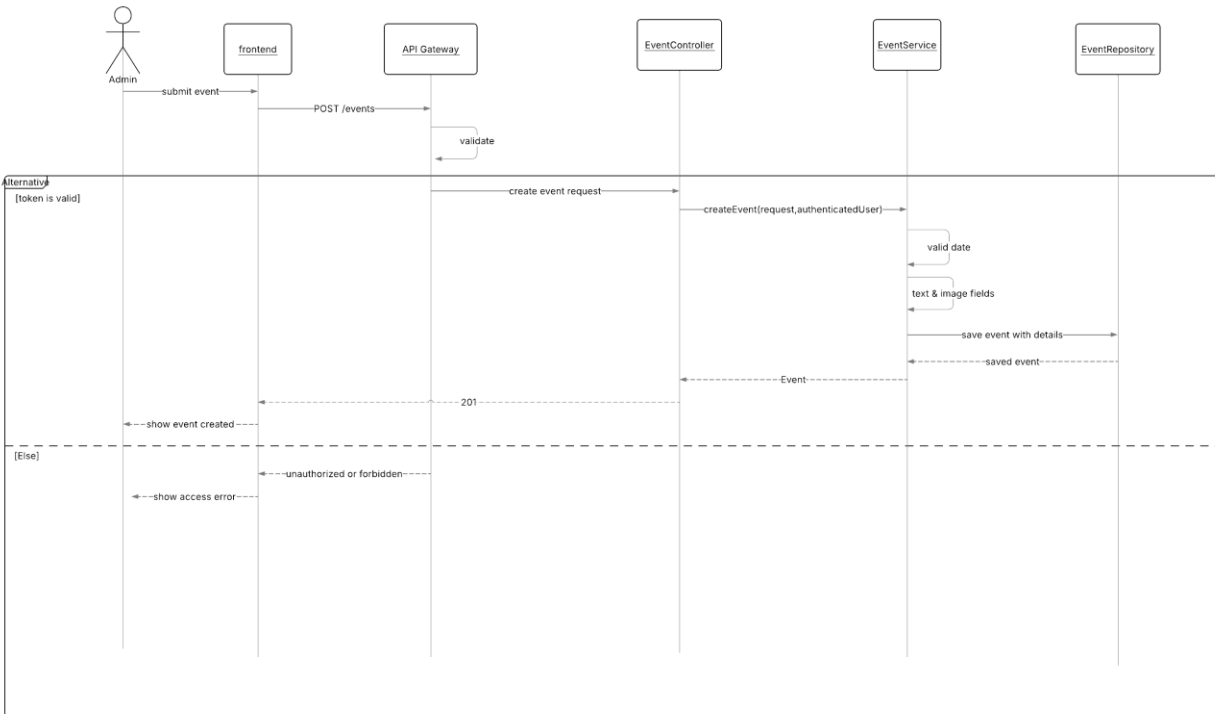
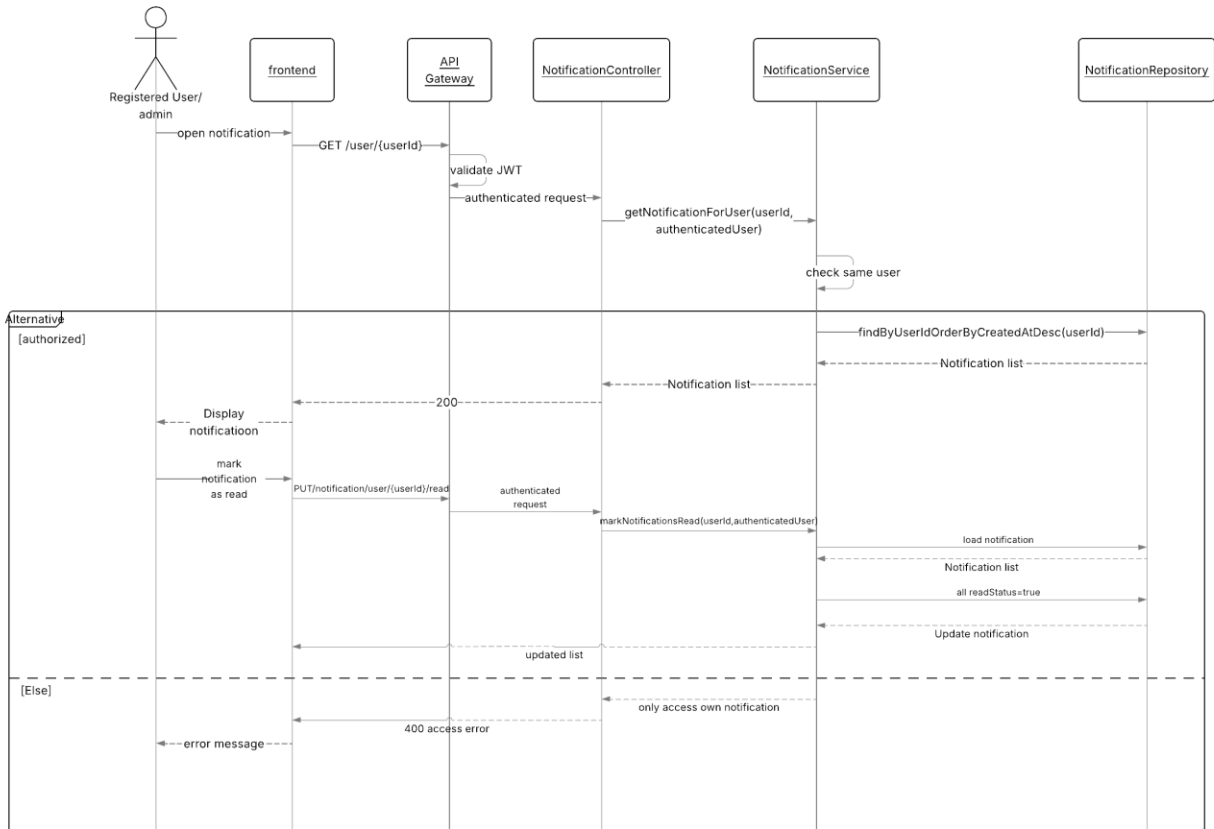


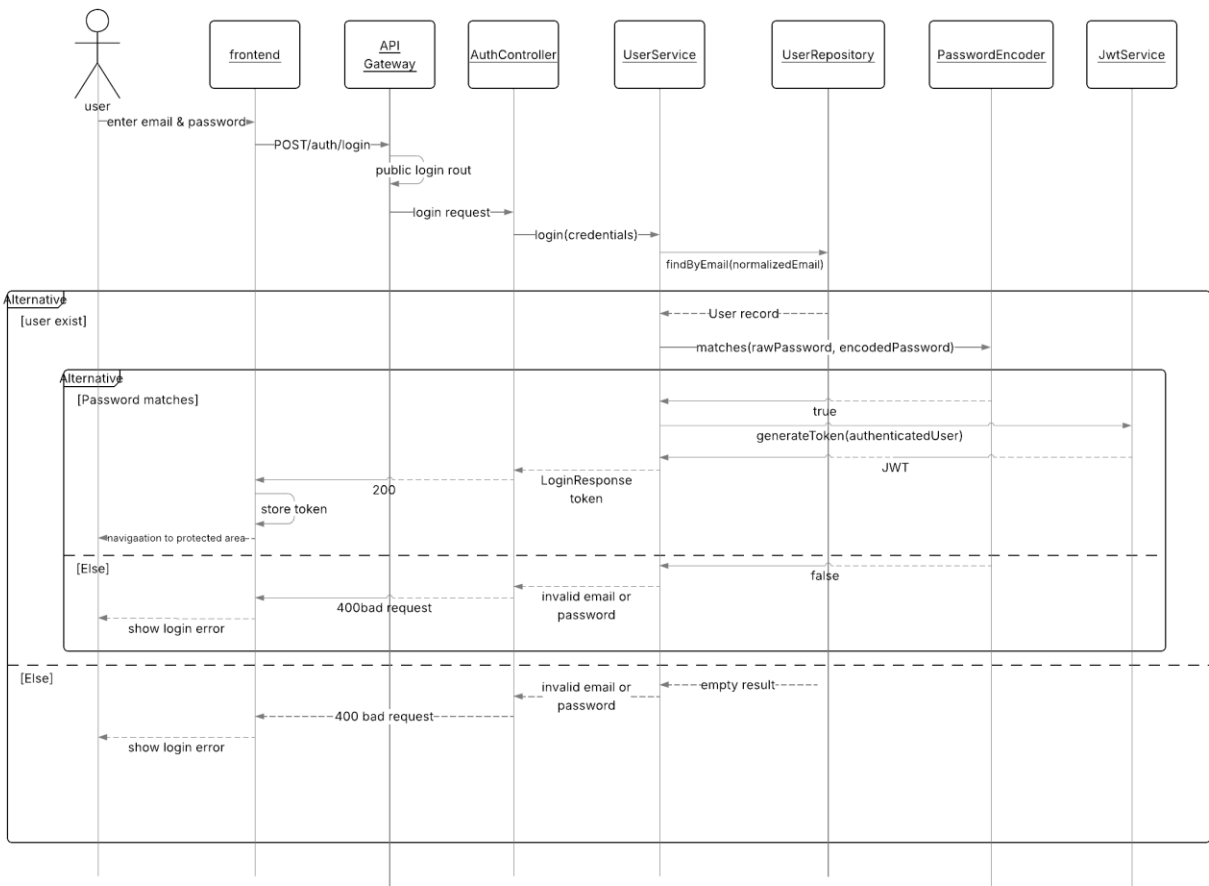
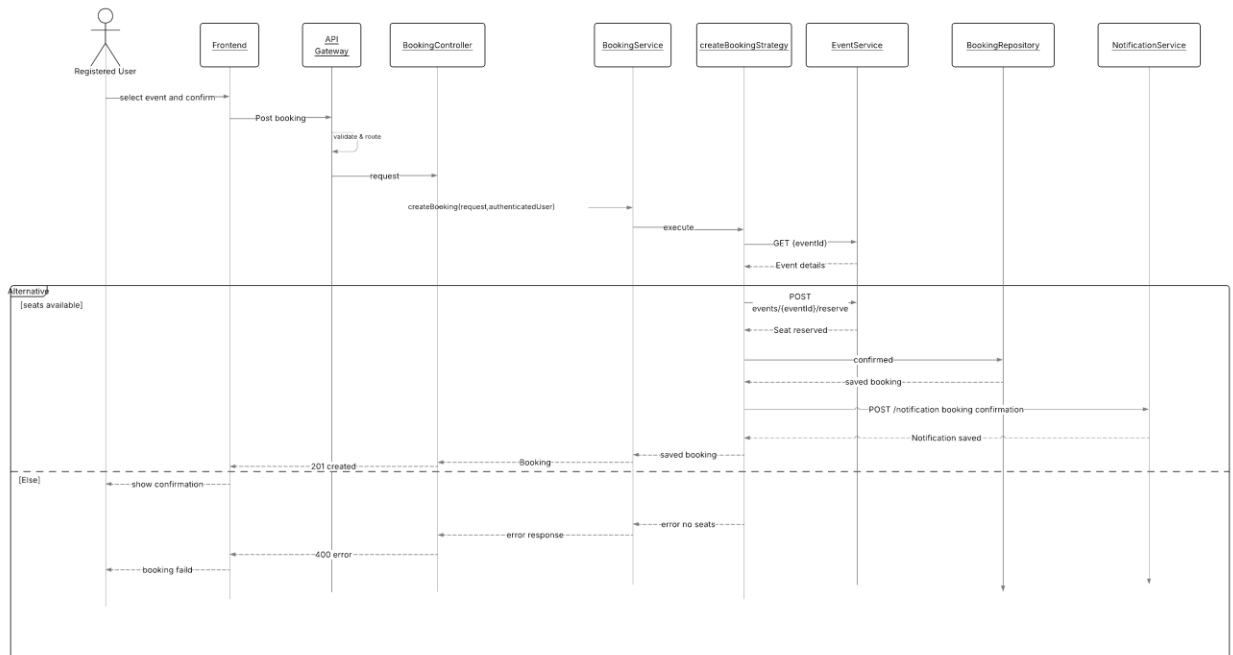


8. Sequence Diagrams

Included Sequence Diagrams

- Authentication Sequence Diagram
- Events Sequence Diagram
- Booking Sequence Diagram
- Notifications Sequence Diagram



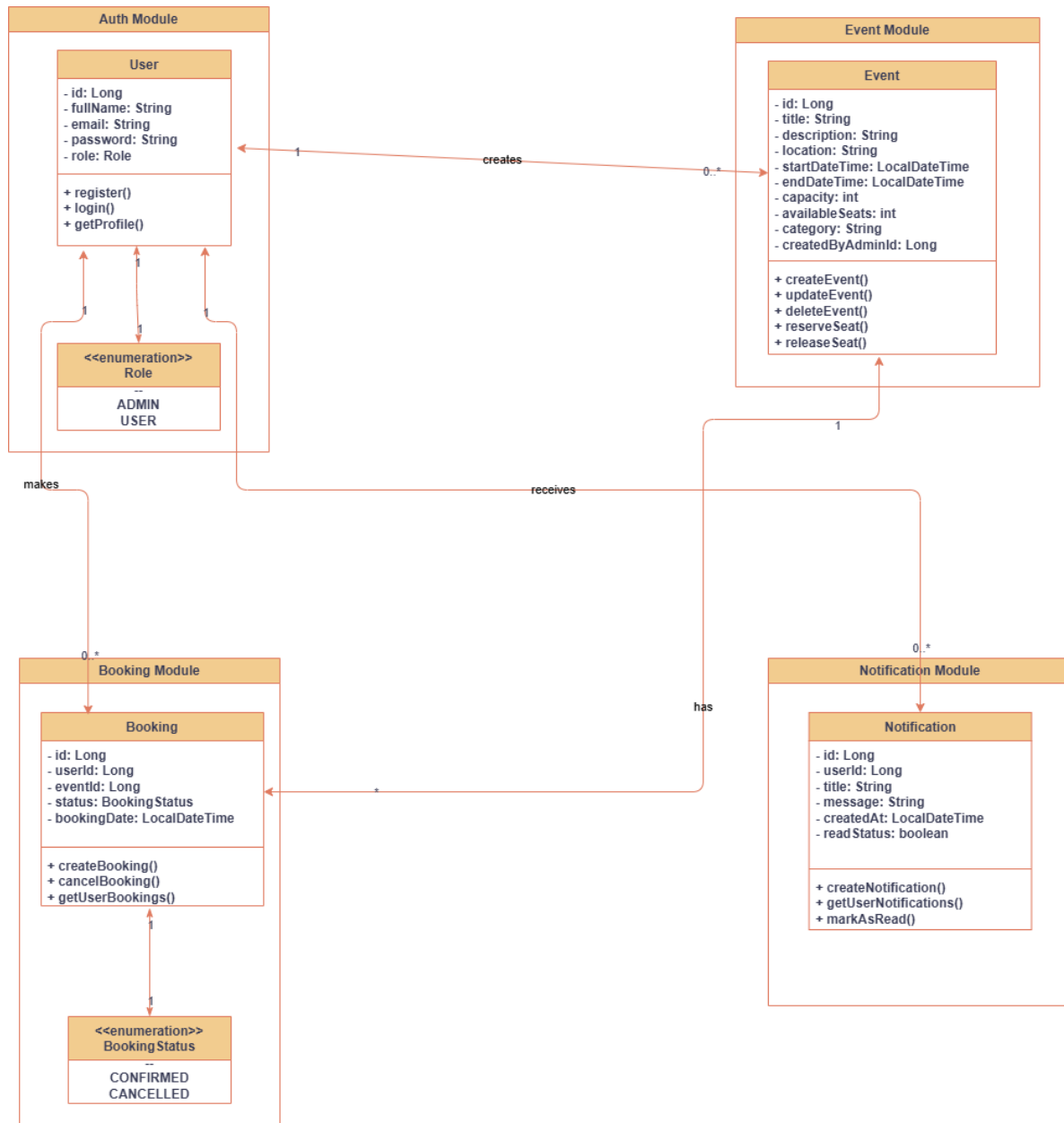


9. Domain Class Diagram

The domain class diagram represents the core business entities and relationships.

Main entities:

- User
- Role
- Event
- Booking
- BookingStatus
- Notification



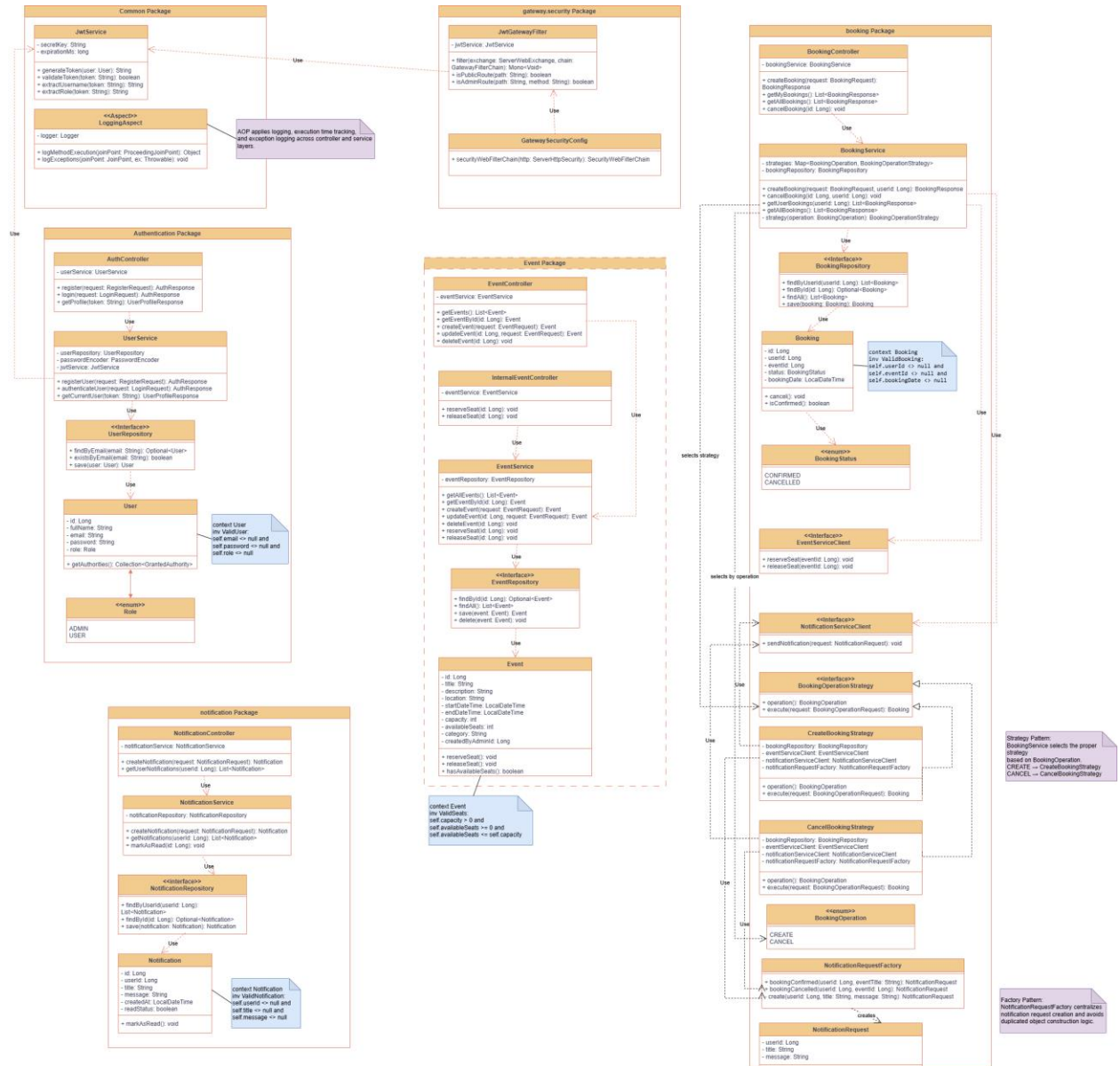
10. Implementation Class Diagram

The implementation class diagram demonstrates:

Packages:

- common

- gateway.security
- authentication
- event
- booking
- notification

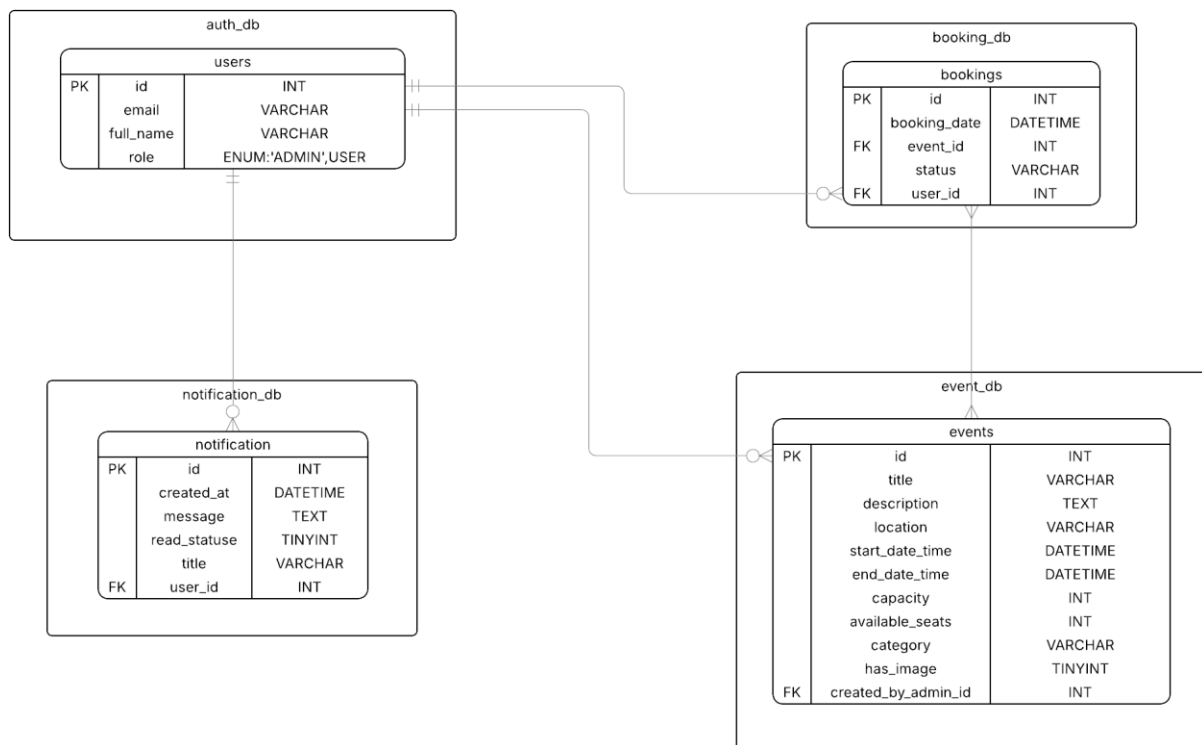


11. Entity Relationship Diagram (ERD)

The ERD describes the database structure and relationships between entities.

Main tables:

- users
- events
- bookings
- notifications



12. Object Constraint Language (OCL)

Object Constraint Language (OCL) is used to formally define business rules and integrity constraints within the Eventify system.

OCL-1 Positive Event Capacity

```
context Event
inv PositiveCapacity:
self.capacity > 0
```

OCL-2 Valid Available Seats

```
context Event
inv ValidAvailableSeats:
self.availableSeats >= 0 and
self.availableSeats <= self.capacity
```

OCL-3 Unique User Email

```
context User
inv UniqueEmail:
User.allInstances()->isUnique(email)
```

OCL-4 Booking Must Reference Event

```
context Booking
inv BookingMustReferenceEvent:
self.eventId <> null
```

OCL-5 Notification Ownership

```
context Notification
inv NotificationMustBelongToUser:
self.userId <> null
```

13. Aspect-Oriented Programming (AOP)

The system uses Aspect-Oriented Programming to separate cross-cutting concerns from business logic.

Implemented AOP Features

- Request logging
- Method execution logging
- Exception logging
- Performance monitoring

AOP Component

LoggingAspect

The aspect intercepts controller and service layer operations.

14. Docker Deployment

The system is containerized using Docker.

Docker Components

- Dockerfiles for each service
- docker-compose.yml
- Isolated service containers
- Independent databases

Docker enables:

- Simplified deployment
- Environment consistency
- Service isolation

15. Clean Code Principles

The project follows clean code principles including:

- Layered architecture
- Single Responsibility Principle (SRP)
- Dependency Inversion Principle (DIP)
- Interface Segregation Principle (ISP)
- Separation of concerns
- Reusable services
- Modular design

16. Design Patterns Used

Design Pattern	Usage
Repository Pattern	Data access abstraction
DTO Pattern	Request/response separation
Strategy Pattern	Booking operation selection
Factory Pattern	Notification request creation
Layered Architecture	Separation of responsibilities
Aspect Pattern	LoggingAspect
Dependency Injection	Spring-managed dependencies

17. Microservices & Cloud Features

The project implements cloud-native microservices architecture.

Implemented Features

- Independent services
- API Gateway
- Eureka Discovery Server
- Distributed architecture
- Service isolation
- Independent databases
- Dockerized deployment

18. Future Enhancements

Possible future improvements include:

- Payment integration
- Email notifications
- Event recommendation system
- Mobile application

20. Conclusion

Eventify successfully demonstrates the implementation of a scalable and secure microservices-based event management platform using modern software engineering practices.